SENTINEL
BUILD ROADMAP

Kubernetes Pod Entropy Monitor

What we are building: A Kubernetes Pod Entropy Monitor that detects container drift and auto-purges compromised pods.

Core Concept: Every container starts with score 100 (pristine). As drift is detected (new files, processes, ports, packages), the score drops. Below threshold = auto-purge.

Total Phases	7
Total Tasks	40+
Estimated Time	5-8 hours
Languages	Go, TypeScript, YAML

PHASE 1: KUBERNETES MANIFESTS

Foundation - What gets deployed and monitored

#	Task	Description	File
1.1	Demo App Namespace	Isolated namespace for test workloads	k8s/demo-app/namespace.yaml
1.2	Nginx Deployment	Web server pods to monitor	k8s/demo-app/nginx-deployment.yaml
1.3	Postgres Deployment	Database pod to monitor	k8s/demo-app/postgres-deployment.yaml
1.4	Sentinel Namespace	Namespace for Sentinel components	k8s/sentinel/namespace.yaml
1.5	ConfigMap	Scoring weights, thresholds, settings	k8s/sentinel/configmap.yaml
1.6	Agent DaemonSet	Runs agent on every node	k8s/sentinel/agent-daemonset.yaml
1.7	API Deployment	Central API server	k8s/sentinel/api-deployment.yaml
1.8	Controller Deployment	Auto-purge controller	k8s/sentinel/controller-deployment.yaml

PHASE 2: ENTROPY AGENT (Go)

The brain - captures baselines and detects drift

#	Task	Description	File
2.1	Go Module Setup	Initialize Go module with dependencies	agent/go.mod
2.2	Baseline Capture	Snapshot filesystem, processes, ports, packages	agent/pkg/baseline/baseline.go
2.3	Drift Monitor	Compare current state vs baseline	agent/pkg/monitor/monitor.go
2.4	Score Calculator	Calculate entropy score (0-100)	agent/pkg/scoring/scoring.go
2.5	API Reporter	Send findings to API server	agent/pkg/reporter/reporter.go
2.6	Agent Main	Entry point, monitoring loop	agent/main.go
2.7	Agent Dockerfile	Container image build	agent/Dockerfile

PHASE 3: API SERVER (Go)

Central hub - stores data, serves UI, exposes endpoints

#	Task	Description	File
3.1	Go Module Setup	Initialize with Fiber, SQLite deps	api/go.mod
3.2	Data Models	Pod, DriftEvent, Baseline, Config structs	api/models/models.go
3.3	SQLite Store	Database layer with CRUD operations	api/store/store.go
3.4	HTTP Handlers	REST endpoints for pods, config, leaderboard	api/handlers/handlers.go
3.5	WebSocket Hub	Real-time score updates to UI	api/websocket/hub.go
3.6	API Main	Server entry point, routes, middleware	api/main.go
3.7	API Dockerfile	Container image build	api/Dockerfile

PHASE 4: PURGE CONTROLLER (Go)

The enforcer - watches scores and kills bad pods

#	Task	Description	File
4.1	Go Module Setup	Initialize with K8s client-go	controller/go.mod
4.2	Purger	Pod deletion logic via K8s API	controller/pkg/purger/purger.go
4.3	Reconciler	Watch scores, enforce thresholds	controller/pkg/reconciler/reconciler.go
4.4	Controller Main	Entry point, reconciliation loop	controller/main.go
4.5	Controller Dockerfile	Container image build	controller/Dockerfile

PHASE 5: REACT UI (TypeScript)

The dashboard - visualize scores, manage pods

#	Task	Description	File
5.1	Project Setup	Vite, React, TypeScript, Tailwind	ui/package.json + configs
5.2	TypeScript Types	Pod, DriftEvent, Config interfaces	ui/src/types/index.ts
5.3	API Client	HTTP client for backend	ui/src/api/client.ts
5.4	React Hooks	Data fetching, WebSocket, actions	ui/src/hooks/index.ts
5.5	Leaderboard	Pod ranking table	ui/src/components/Leaderboard/
5.6	PodDetail	Individual pod view with events	ui/src/components/PodDetail/
5.7	PodManager	Create/delete pods	ui/src/components/PodManager/
5.8	PurgeConfig	Configure auto-purge settings	ui/src/components/PurgeConfig/
5.9	Charts	Score visualizations	ui/src/components/Charts/
5.10	Main App	Layout, routing, state	ui/src/App.tsx
5.11	UI Dockerfile	Container image build	ui/Dockerfile

PHASE 6: SCRIPTS AND DEVTOOLS

Automation for development and testing

#	Task	Description	File
6.1	Makefile	Build, test, deploy commands	Makefile
6.2	Docker Compose	Local testing without K8s	docker-compose.yml
6.3	Tiltfile	Hot-reload K8s development	Tiltfile
6.4	Setup Minikube	Cluster bootstrap script	scripts/setup-minikube.sh
6.5	Deploy All	Full stack deployment	scripts/deploy-all.sh
6.6	Simulate Attack	Test drift detection	scripts/simulate-attack.sh
6.7	Git Config	.gitignore, README	.gitignore, README.md

PHASE 7: INTEGRATION AND TESTING

Wire everything together

#	Task	Description
7.1	Build Docker Images	Build all 4 component images
7.2	Deploy to K8s	Apply all manifests
7.3	Verify Agent	Check baseline capture working
7.4	Verify API	Test endpoints with curl
7.5	Verify UI	Open dashboard, check data
7.6	Run Attack Simulation	Watch scores drop
7.7	Verify Auto-Purge	Confirm bad pods get killed

SCORING SYSTEM

Formula: ENTROPY_SCORE = 100 - Sum(category_weight x drift_severity)

Category Weights:

Category	Weight	What it monitors
FILESYSTEM	35%	New files in /bin, /tmp, config changes
PROCESSES	30%	New processes, unexpected parent-child
NETWORK	20%	New listening ports, outbound connections
PACKAGES	10%	apt/apk/yum installs
PERMISSIONS	5%	New users, sudo escalation

Severity Points:

Event	Points
New executable file	+15
Modified system binary	+25
New running process	+10
New listening port	+20
New installed package	+15
New user created	+30
Config file modified	+5

API ENDPOINTS REFERENCE

Method	Endpoint	Description
GET	`/api/pods`	List all pods with scores
GET	`/api/pods/:id`	Get pod details + drift events
POST	`/api/pods`	Create new pod (triggers K8s)
DELETE	`/api/pods/:id`	Delete/purge pod
POST	`/api/pods/:id/report`	Receive agent report
GET	`/api/pods/:id/baseline`	Get baseline snapshot
POST	`/api/pods/:id/baseline`	Reset baseline
GET	`/api/leaderboard`	Sorted pods by score
GET	`/api/config`	Get purge configuration
PUT	`/api/config`	Update purge config
WS	`/api/ws/scores`	Real-time score stream
GET	`/health`	Health check
GET	`/ready`	Readiness check

PURGE SPEED SETTINGS

Speed	Threshold	Behavior
Off	-	No auto-purge, manual only
Conservative	<30	Alert, wait 5 min, re-check, then purge
Moderate	<40	Alert, wait 1 min, then purge
Aggressive	<50	Immediate purge + replace

BUILD CHECKLIST

Phase 1: Kubernetes Manifests

- [ ] 1.1 Demo App Namespace
- [ ] 1.2 Nginx Deployment
- [ ] 1.3 Postgres Deployment
- [ ] 1.4 Sentinel Namespace
- [ ] 1.5 ConfigMap
- [ ] 1.6 Agent DaemonSet
- [ ] 1.7 API Deployment
- [ ] 1.8 Controller Deployment

Phase 2: Entropy Agent

- [ ] 2.1 Go Module Setup
- [ ] 2.2 Baseline Capture
- [ ] 2.3 Drift Monitor
- [ ] 2.4 Score Calculator
- [ ] 2.5 API Reporter
- [ ] 2.6 Agent Main
- [ ] 2.7 Agent Dockerfile

Phase 3: API Server

- [ ] 3.1 Go Module Setup
- [ ] 3.2 Data Models
- [ ] 3.3 SQLite Store
- [ ] 3.4 HTTP Handlers
- [ ] 3.5 WebSocket Hub
- [ ] 3.6 API Main
- [ ] 3.7 API Dockerfile

Phase 4: Purge Controller

- [ ] 4.1 Go Module Setup
- [ ] 4.2 Purger
- [ ] 4.3 Reconciler
- [ ] 4.4 Controller Main
- [ ] 4.5 Controller Dockerfile

Phase 5: React UI

- [ ] 5.1 Project Setup
- [ ] 5.2 TypeScript Types
- [ ] 5.3 API Client

- [ ] 5.4 React Hooks
- [ ] 5.5 Leaderboard
- [ ] 5.6 PodDetail
- [ ] 5.7 PodManager
- [ ] 5.8 PurgeConfig
- [ ] 5.9 Charts
- [ ] 5.10 Main App
- [ ] 5.11 UI Dockerfile

Phase 6: Scripts

- [ ] 6.1 Makefile
- [ ] 6.2 Docker Compose
- [ ] 6.3 Tiltfile
- [ ] 6.4 Setup Minikube
- [ ] 6.5 Deploy All
- [ ] 6.6 Simulate Attack
- [ ] 6.7 Git Config

Phase 7: Integration

- [ ] 7.1 Build Images
- [ ] 7.2 Deploy to K8s
- [ ] 7.3 Verify Agent
- [ ] 7.4 Verify API
- [ ] 7.5 Verify UI
- [ ] 7.6 Attack Simulation
- [ ] 7.7 Verify Auto-Purge